



Aula 6 — Struct e Typedef em C

Até agora, aprendemos a trabalhar com variáveis, arrays, funções, condições e loops.

Mas existe uma situação que começa a aparecer quando fazemos projetos maiores.

Imagine que queremos guardar os dados de um aluno.

Precisamos do:

```
Nome  
Idade  
Nota
```

Podemos criar

```
char nome[50000];  
int idade;  
float nota;
```

Até aqui tudo bem.

Mas agora imagine que temos 50 alunos.

Começamos a ter várias variáveis espalhadas pelo programa.

E se quisermos representar um aluno como uma única coisa?

É aqui que entra o `struct`.

1. O que é uma struct?

`struct` permite criar um tipo de dado que pode guardar diferentes informações relacionadas.

Pensa em uma ficha de cadastro.

Uma ficha de aluno pode ter:

```
-----  
ALUNO  
-----  
Nome: José  
Idade: 18
```

Nota: 15.5

Em vez de tratar essas informações como coisas completamente separadas, podemos agrupá-las.

Em C:

```
struct Aluno {  
    char nome[50];  
    int idade;  
    float nota;  
};
```

Acabamos de criar uma estrutura chamada `Aluno`.

Ela possui três informações:

```
nome  
idade  
nota
```

2. Criando uma variável do tipo struct

Depois de criar a estrutura, podemos criar um aluno:

```
struct Aluno aluno1;
```

Agora `aluno1` possui:

```
aluno1.nome  
aluno1.idade  
aluno1.nota
```

Para acessar cada informação, usamos o ponto `..`

Por exemplo:

```
aluno1.idade = 18;  
aluno1.nota = 15.5;
```

E podemos mostrar:

```
printf("Idade: %d\n", aluno1.idade);  
printf("Nota: %.2f\n", aluno1.nota);
```

3. Um exemplo completo

Vamos colocar isso em um programa:

```
#include <stdio.h>  
  
struct Aluno {  
    char nome[50];  
    int idade;  
    float nota;  
};  
  
int main() {  
  
    struct Aluno aluno1;  
  
    printf("Digite o nome: ");  
    scanf("%49s", aluno1.nome);  
  
    printf("Digite a idade: ");  
    scanf("%d", &aluno1.idade);  
  
    printf("Digite a nota: ");  
    scanf("%f", &aluno1.nota);  
  
    printf("\n=== DADOS DO ALUNO ===\n");  
  
    printf("Nome: %s\n", aluno1.nome);  
    printf("Idade: %d\n", aluno1.idade);  
    printf("Nota: %.2f\n", aluno1.nota);  
  
    return 0;  
}
```

Uma possível execução:

```
Digite o nome: Jose  
Digite a idade: 18  
Digite a nota: 15.5
```

```
=== DADOS DO ALUNO ===  
Nome: Jose
```

Idade: 18
Nota: 15.50

Agora temos uma coisa interessante.

Em vez de pensar:

```
nome é uma coisa  
idade é outra  
nota é outra
```

podemos pensar:

```
aluno1  
├─ nome  
├─ idade  
└─ nota
```

Esse é um dos objetivos de uma `struct`.

4. Podemos ter vários alunos

Agora imagine que queremos cadastrar vários alunos.

Lembra dos arrays?

Podemos juntar arrays com `struct`.

Por exemplo:

```
struct Aluno alunos[3];
```

Isso significa que temos um array capaz de guardar 3 alunos.

Podemos acessar:

```
alunos[0]  
alunos[1]  
alunos[2]
```

E cada aluno possui:

```
nome
idade
nota
```

Podemos utilizar um `for`:

```
#include <stdio.h>

struct Aluno {
    char nome[50];
    int idade;
    float nota;
};

int main() {

    struct Aluno alunos[3];

    for (int i = 0; i < 3; i++) {

        printf("\nAluno %d\n", i + 1);

        printf("Nome: ");
        scanf("%49s", alunos[i].nome);

        printf("Idade: ");
        scanf("%d", &alunos[i].idade);

        printf("Nota: ");
        scanf("%f", &alunos[i].nota);
    }

    printf("\n=== ALUNOS CADASTRADOS ===\n");

    for (int i = 0; i < 3; i++) {

        printf("\nAluno %d\n", i + 1);
        printf("Nome: %s\n", alunos[i].nome);
        printf("Idade: %d\n", alunos[i].idade);
        printf("Nota: %.2f\n", alunos[i].nota);
    }

    return 0;
}
```

Aqui estamos juntando três conceitos:

```
struct
  ↓
organiza os dados do aluno

array
  ↓
guarda vários alunos

for
  ↓
percorre os alunos
```

Já começa a parecer um pequeno sistema de cadastro.

5. Agora entra o typedef

Você provavelmente percebeu que toda vez precisamos escrever:

```
struct Aluno
```

Por exemplo:

```
struct Aluno aluno1;
```

E:

```
struct Aluno alunos[10];
```

Não seria mais simples escrever apenas:

```
Aluno aluno1;
```

É justamente aí que podemos utilizar `typedef`.

O `typedef` permite criar um nome alternativo para um tipo.

Podemos fazer:

```
typedef struct {
    char nome[50];
    int idade;
```

```
float nota;  
} Aluno;
```

Agora podemos criar uma variável simplesmente assim:

```
Aluno aluno1;
```

Muito mais limpo.

6. Struct + typedef

Vamos comparar.

Sem `typedef`:

```
struct Aluno {  
    char nome[50];  
    int idade;  
    float nota;  
};  
  
struct Aluno aluno1;
```

Com `typedef`:

```
typedef struct {  
    char nome[50];  
    int idade;  
    float nota;  
} Aluno;  
  
Aluno aluno1;
```

Os dois funcionam.

O `typedef` apenas nos permite dar um nome mais simples ao tipo.

7. Um exemplo completo usando typedef

```
#include <stdio.h>  
  
typedef struct {
```

```

    char nome[50];
    int idade;
    float nota;
} Aluno;

int main() {

    Aluno aluno;

    printf("Digite o nome: ");
    scanf("%49s", aluno.nome);

    printf("Digite a idade: ");
    scanf("%d", &aluno.idade);

    printf("Digite a nota: ");
    scanf("%f", &aluno.nota);

    printf("\n=== ALUNO ===\n");
    printf("Nome: %s\n", aluno.nome);
    printf("Idade: %d\n", aluno.idade);
    printf("Nota: %.2f\n", aluno.nota);

    return 0;
}

```

Agora fica mais fácil de ler.

Quando vemos:

```
Aluno aluno;
```

podemos entender:

“Estou criando uma variável que representa um aluno.”

8. Struct não precisa ser apenas para alunos

E aqui começa a parte divertida.

Podemos criar estruturas para praticamente qualquer coisa.

Por exemplo, um produto:

```

typedef struct {
    char nome[50];

```



```
float preco;  
int quantidade;  
} Produto;
```

Agora podemos criar:

```
Produto produto1;
```

E preencher:

```
produto1.preco = 2500;  
produto1.quantidade = 5;
```

Outro exemplo: funcionário.

```
typedef struct {  
    char nome[50];  
    int idade;  
    float salario;  
} Funcionario;
```

Ou uma conta:

```
typedef struct {  
    char titular[50];  
    float saldo;  
} Conta;
```

A ideia é sempre a mesma:

pegamos várias informações relacionadas e agrupamos em uma única estrutura.

9. Struct com funções

Agora vamos juntar o que aprendemos na aula anterior.

Podemos passar uma `struct` para uma função.

Por exemplo:

```
#include <stdio.h>  
  
typedef struct {
```

```

    char nome[50];
    int idade;
    float nota;
} Aluno;

void mostrarAluno(Aluno aluno) {

    printf("\n=== DADOS ===\n");
    printf("Nome: %s\n", aluno.nome);
    printf("Idade: %d\n", aluno.idade);
    printf("Nota: %.2f\n", aluno.nota);
}

int main() {

    Aluno aluno;

    printf("Nome: ");
    scanf("%49s", aluno.nome);

    printf("Idade: ");
    scanf("%d", &aluno.idade);

    printf("Nota: ");
    scanf("%f", &aluno.nota);

    mostrarAluno(aluno);

    return 0;
}

```

Aqui temos:

```

struct
  ↓
cria o modelo do aluno

typedef
  ↓
facilita o nome do tipo

função
  ↓
mostra os dados

main
  ↓
organiza o programa

```

É exatamente assim que começamos a organizar programas maiores.

10. Projeto — Sistema simples de alunos

Agora vamos fazer um pequeno projeto juntando vários conceitos que aprendemos.

O programa vai cadastrar 3 alunos e mostrar os dados deles.

```
#include <stdio.h>

typedef struct {
    char nome[50];
    int idade;
    float nota;
} Aluno;

void mostrarAluno(Aluno aluno) {

    printf("\n-----\n");
    printf("Nome: %s\n", aluno.nome);
    printf("Idade: %d\n", aluno.idade);
    printf("Nota: %.2f\n", aluno.nota);

    if (aluno.nota >= 10) {
        printf("Situacao: Aprovado\n");
    } else {
        printf("Situacao: Reprovado\n");
    }
}

int main() {

    Aluno alunos[3];

    printf("=== CADASTRO DE ALUNOS ===\n");

    for (int i = 0; i < 3; i++) {

        printf("\nAluno %d\n", i + 1);

        printf("Nome: ");
        scanf("%49s", alunos[i].nome);

        printf("Idade: ");
        scanf("%d", &alunos[i].idade);

        printf("Nota: ");
        scanf("%f", &alunos[i].nota);
    }
```

```

printf("\n=== RESULTADOS ===\n");

for (int i = 0; i < 3; i++) {
    mostrarAluno(alunos[i]);
}

return 0;
}

```

Olha quantas coisas já estão trabalhando juntas:

```

Variáveis
↓
guardam informações

Array
↓
guarda vários alunos

Struct
↓
organiza os dados de cada aluno

Typedef
↓
facilita a criação do tipo Aluno

For
↓
repete o cadastro

Função
↓
mostra os dados

If / else
↓
verifica aprovação

```

Isso já é muito diferente daquele primeiro:

```
printf("Ola, mundo!");
```

E é exatamente esse o objetivo do curso.

Começamos com coisas pequenas e, pouco a pouco, vamos juntando essas ferramentas para construir programas maiores.

Desafio da aula

Agora tente criar uma estrutura chamada `Produto`.

Ela deve possuir:

```
Nome  
Preço  
Quantidade
```

Utilize:

```
typedef struct
```

Depois crie um array para armazenar 5 produtos.

O programa deve:

1. Cadastrar os 5 produtos.
2. Mostrar todos os produtos.
3. Mostrar o preço de cada produto.
4. Mostrar a quantidade disponível.
5. Criar uma função para mostrar os dados de um produto.

Se quiser deixar o desafio um pouco mais difícil, crie também uma função que calcule o valor total do estoque:

```
preço × quantidade
```

Por exemplo:

```
Produto: Teclado  
Preço: 15.000 Kz  
Quantidade: 3  
  
Valor do estoque: 45.000 Kz
```

O que aprendemos nesta aula

Nesta aula aprendemos:

- O que é uma `struct`;
- Como criar uma estrutura;
- Como criar variáveis usando uma `struct`;

- Como acessar os dados usando `.;`
- Como criar arrays de `struct`;
- O que é `typedef`;
- Como utilizar `typedef` com `struct`;
- Como passar uma `struct` para uma função;
- Como combinar `struct`, arrays, funções, loops e condicionais;
- Como criar um pequeno sistema de cadastro.

A partir daqui, o C começa a ficar muito mais interessante.

Já não estamos apenas trabalhando com números e textos separados.

Agora conseguimos criar nossos próprios tipos de dados para representar coisas do mundo real.

E isso abre caminho para projetos muito maiores.

